
ICPC Algorithm Library

目录

1 Basic	1
1.1 test	1
1.2 pai	1
1.3 run	1
1.4 缺省源	1
1.5 随机数	1
1.6 Matrix	1
2 Graph	1
2.1 Dijkstra	1
2.2 Tarjan SCC	2
2.3 2-SAT	2
3 Geometry	2
3.1 vector	2
3.2 seg_poly	3

1 Basic

1.1 test

```
export fs="-fsanitize=address,undefined"
ulimit -s 1024000
ulimit -v 1024000
mk() { g++ -o $1 $1.cpp -O2; }
```

1.2 pai

```
pai() {
    mk a; mk bf; mk gen
    while true; do
        ./gen > 1.in
        ./bf < 1.in > 1.ans
        ./a < 1.in > 1.out
        if diff -Zq 1.out 1.ans; then echo ac; else echo wa; break; fi
    done
}
```

1.3 run

```
shopt -s nullglob # 没有匹配到任何文件时，让它返回空
ulimit -s 1024000
g++ $1.cpp -o $1 -std=c++20 -O2 -fsanitize=address,undefined || exit
for f in "$2"/*.in; do
    x=${f%.in}
    \time -f "$x: %es %MKB" ./$1 < $x.in > $1.out
    diff -Zq $1.out $x.ans
done
```

1.4 缺省源

```
#include <bits/stdc++.h>
using namespace std;

#define ALL(s) s.begin(), s.end()
#define SZ(s) int(s.size())
#define pb push_back

using LL = long long;
using ULL = unsigned long long;
using I = __int128;

void Cas() {
}

int main() {
```

```
cin.tie(0)->sync_with_stdio(0);
int t = 1;
cin >> t;
while (t--) {
    Cas();
}
return 0;
}
```

1.5 随机数

```
ULL seed = chrono::steady_clock::now().time_since_epoch().count();
mt19937_64 rnd(seed);
```

1.6 Matrix

```
template <class T>
struct Mat {
    int n, m;
    vector<T> a;
    Mat(int n = 0, int m = 0, T v = {}) : n(n), m(m), a(n * m, v) {}
    T& operator()(int i, int j) { return a[i * m + j]; }
};

using M = Mat<LL>;
M mul(M a, M b) {
    M c(a.n, b.m);
    for (int i = 0; i < a.n; i++) {
        for (int j = 0; j < b.m; j++) {
            for (int k = 0; k < a.m; k++) {
                c(i, j) = max(c(i, j), a(i, k) + b(k, j));
            }
        }
    }
    return c;
}
```

2 Graph

2.1 Dijkstra

```
struct Node {
    int u;
    LL d;
    bool operator<(const Node& o) const {
        return d > o.d;
    }
}
```

```
};
priority_queue<Node> q;
vector<LL> D(n, INF);
D[0] = 0;
q.push({0, 0});
while (SZ(q)) {
    auto [u, d] = q.top();
    q.pop();
    if (d != D[u]) continue;
    for (auto [v, w] : G[u]) {
        if (D[v] > d + w) {
            D[v] = d + w;
            q.push({v, D[v]});
        }
    }
}
}
```

2.2 Tarjan SCC

```
struct SCC {
    int n, tim = 0, scc = 0;
    vector<vector<int>> G;
    vector<int> dfn, low, bel, stk;
    SCC(int N) : n(N), G(N), dfn(N), low(N), bel(N, -1) {}
    void add(int u, int v) { G[u].push_back(v); }
    void dfs(int u) {
        dfn[u] = low[u] = ++tim;
        stk.push_back(u);
        for (int v : G[u]) {
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (bel[v] == -1) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]) return;
        while (1) {
            int v = stk.back();
            stk.pop_back();
            bel[v] = scc;
            if (v == u) break;
        }
        scc++;
    }
    void run() {
        for (int u = 0; u < n; u++) {
            if (!dfn[u]) dfs(u);
        }
    }
};
```

```
}
};
```

2.3 2-SAT

每个布尔变量 u 拆成两个点 u_0, u_1 ，其中 u_a 表示 $u = a$ ，代码中可令 $u_a = 2u + a$ 。 u_a 的相反状态是 $u_{a \oplus 1}$ 。

所有限制最终都化成析取式。对 $u_a \vee v_b$ ，加入两条边

$$u_{a \oplus 1} \rightarrow v_b, \quad v_{b \oplus 1} \rightarrow u_a.$$

常见限制的建边方式如下：

- $u_a \Rightarrow v_b$ ：加入 $u_a \rightarrow v_b$ 和 $v_{b \oplus 1} \rightarrow u_{a \oplus 1}$ 。
- 强制 $u = a$ ：加入 $u_{a \oplus 1} \rightarrow u_a$ 。
- 禁止 $u = a$ 与 $v = b$ 同时成立：加入 $u_a \rightarrow v_{b \oplus 1}$ 和 $v_b \rightarrow u_{a \oplus 1}$ 。
- $u = v$ ：加入 $u_0 \leftrightarrow v_0$ 和 $u_1 \leftrightarrow v_1$ 。
- $u \neq v$ ：加入 $u_0 \leftrightarrow v_1$ 和 $u_1 \leftrightarrow v_0$ 。

其中每个 $\leftrightarrow$ 都表示正反两条有向边。

建图后运行 Tarjan。若存在 u 满足 u_0, u_1 在同一强连通分量中，则无解。

前一节的 Tarjan 按分量出栈顺序从小到大编号，因此缩点后的边总是从较大编号指向较小编号。构造方案时，对每个 u 选择强连通分量编号较小的状态：

$$\text{ans}_u = [\text{bel}_{u_0} > \text{bel}_{u_1}].$$

3 Geometry

3.1 vector

```
using db = long double;
struct p2 {
    db x, y;
    db len() { return hypot(x, y); }
    p2 operator+(p2 b) { return {x + b.x, y + b.y}; }
    p2 operator-(p2 b) { return {x - b.x, y - b.y}; }
    p2 operator*(db k) { return {x * k, y * k}; }
    p2 operator/(db k) { return {x / k, y / k}; }
    db operator*(p2 b) { return x * b.x + y * b.y; }
    db operator%(p2 b) { return x * b.y - y * b.x; }
};
```

```
using ps = vector<p2>;
```

3.2 seg_poly

```
// pa 到 pb 的叉积，逆时针转为正  
db side(p2 p, p2 a, p2 b) { return (a - p) % (b - p); }  
db tol(p2 p, p2 a, p2 b) { return abs(side(p, a, b)) / (b - a).len(); }
```